Development

Manage the complexity that's left once AI is writing most of the SQL

dbt config

The config block every model configures itself through

```
{{ config(
    materialized='table',
    tags=['finance'],
    meta={'owner': 'finance-team'}
) }}
```

DRY SQL

A repeated block earns a macro; one or two copies don't

3+ repeats	Worth a macro
1-2 repeats	Leave it inline
Comment the why	Not the what

Jinja whitespace control

Keep compiled SQL readable, not a wall of blank lines

```
{%- set methods = ['card', 'cash'] -%}
{% for m in methods -%}
  sum(case when payment_method = '{{ m }}'
    then amount else 0 end) as {{ m }}
{% endfor %}
```

Reusable macro

One version everyone calls, not five hand-rolled copies

```
{% macro cents_to_dollars(col, precision=2) %}
    round({{ col }} / 100.0, {{ precision }})
{% endmacro %}
```

dbt packages

Open source and narrow in scope, lower risk than building it yourself

```
packages:
  - package: dbt-labs/dbt_utils
    version: [">=1.3.0", "<2.0.0"]
```

dbt deps	Installs them

Materializations

The build strategy a model starts with, before it needs to change

view	Recomputed every query
table	Rebuilt in full
incremental	Only new/changed rows
ephemeral	Inlined CTE, never built

Incremental model

Only new or changed rows get processed after the first run

```
{{ config(
    materialized='incremental',
    unique_key='order_id',
    incremental_strategy='merge'
) }}
select * from {{ ref('stg_orders') }}
{% if is_incremental() %}
where updated_at >= (
    select max(updated_at) from {{ this }}
)
{% endif %}
```

Incremental strategy

How the update actually happens underneath unique_key

append	No unique_key, just inserts
merge	Update matched, insert new
delete+insert	Fallback when no merge
insert_overwrite	Replaces whole partitions

Escape hatch

Rebuild from scratch when the incremental logic itself changed

```
dbt run -s fct_orders --full-refresh
```

Rebuild from scratch

Platform configs

Physical tuning on top of the four materializations

cluster_by	Snowflake
partition_by / cluster_by	BigQuery
dist / sort	Redshift
file_format / liquid clustering	Databricks